1조 기말 프로젝트
Latent Vector Clustering Based on Disentangled Feature forming DAE

Tsoy Alina Vladimirovna

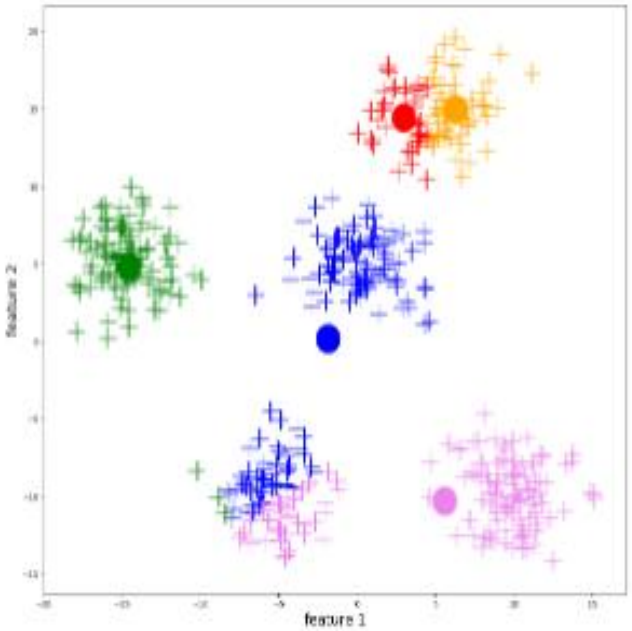
Jeonghan Kong

Woochan Jeong

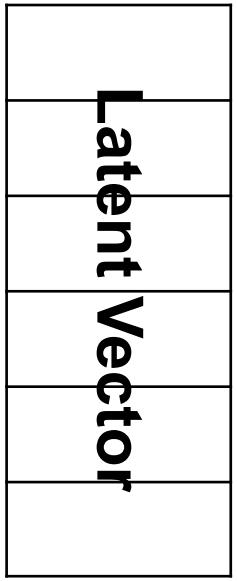
Zaripov Damir

프로젝트 주제 선정 배경 및 동기

기존 주제의 문제점



k

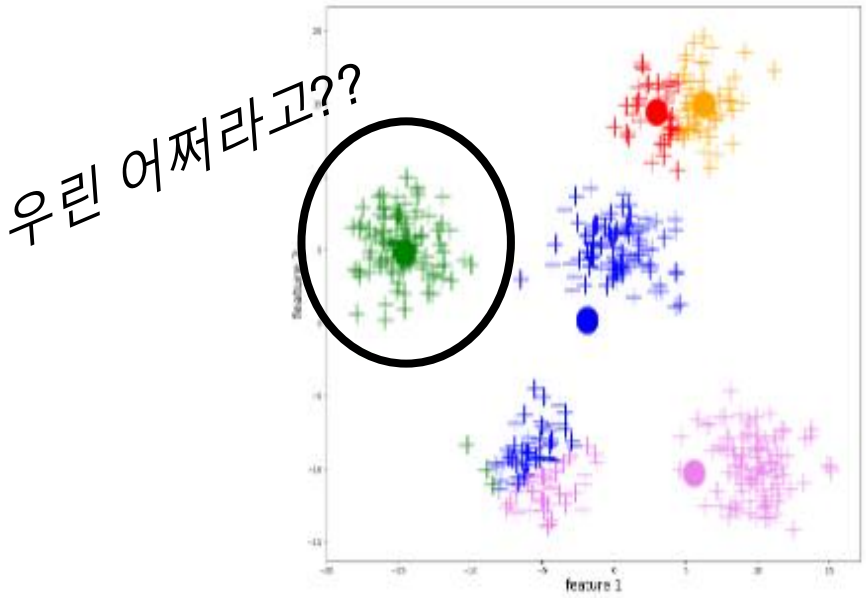


기존의 방식: Clustering에서 가장 효율적으로 나온 k를 latent vector의 크기로 정하여보자.

문제점 : 모델이 너무 뻔하다! 그냥 수업 시간에 나온 모델 2개의 구현!

우리가 의미를 담고자 하는 것을 담지 못 했다!

기존 주제의 문제점



(저번에 얘기는 못 했지만)

진정 하고자 했던 것: 이런 식으로 하다가는 Clustering된 것들이 의미가 없어진다!

Clustering에서 가장 의미가 있는 “k”를 만들었더니,
정작 Latent Vector에는 그 “k”차원이 되더라도,
그저 모델이 k차원 가지고 알아서 학습을 할 뿐.

클러스터가 어떻게 되었는지는 하나도 쓰이지 않고,
우리가 할 수 있는건 진짜 클러스터된 것처럼 Embedding되나 보는 것 뿐!!

클러스터가 어떻게 되었는지도 Embedding에 의미부여할 방법이 없나?

기존 주제의 문제점

그러나 이 방식도...

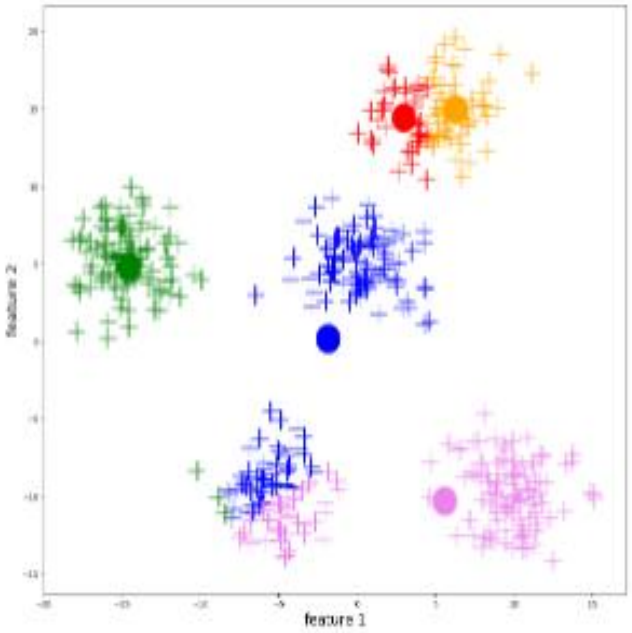
어떻게 Embedding Vector에 직접 영향을 주느냐?

Clustering k의 크기가 기존의 Hidden Dimension에 비해 택도 없이 작다!

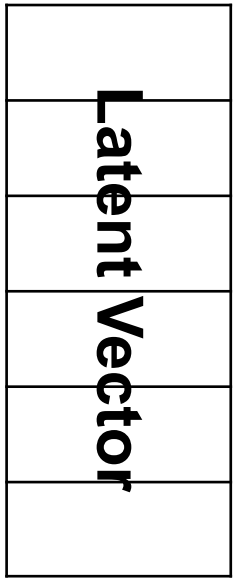
Clustering도 사실 완전한 정보가 아닌데, Dimension도 작아서
Latent Vector의 질이 너무 안 좋다!

해결해야 할 문제가 너무 많고,
해결하더라도 좋은 모델이 아니다!

새로운 주제



k



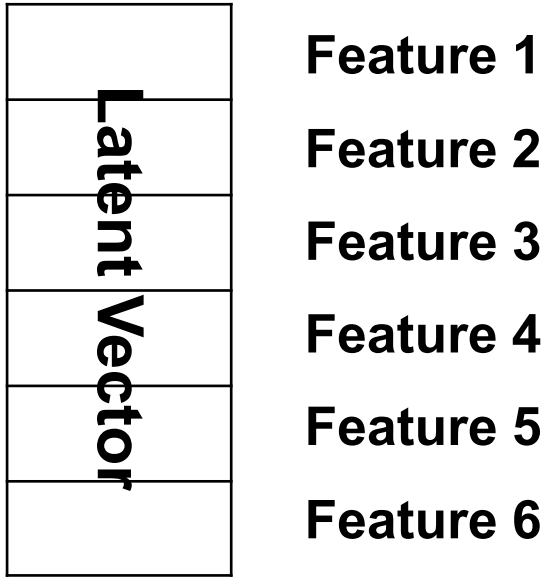
그렇다면 반대로 생각해서 **Latent Vector를 가지고 클러스터링을 해보는 것은 어떨까??**

기존 의문점: Clustering을 이용하면 더 의미 있는 Embedding을 할 수 있지 않을까??

새로운 의문점: Embedding을 이용하면 더 의미 있는 Clustering을 할 수 있지 않을까??

프로젝트 목표 및 문제 정의

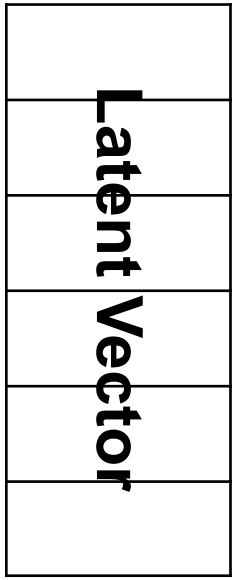
목표/문제 정의 전에 좀 더 살펴봐야할 것



일단 당장 생각한 것은
Latent Vector를 대상으로 클러스터링 해보자.
Latent Vector를 하나의 좌표로 생각해서 전부 클러스터 하면 어떻게 될까??

목표/문제 정의 전에 좀 더 살펴봐야할 것

물론 문제점이 있다!



Feature 1

Feature 2

Feature 3

Feature 4

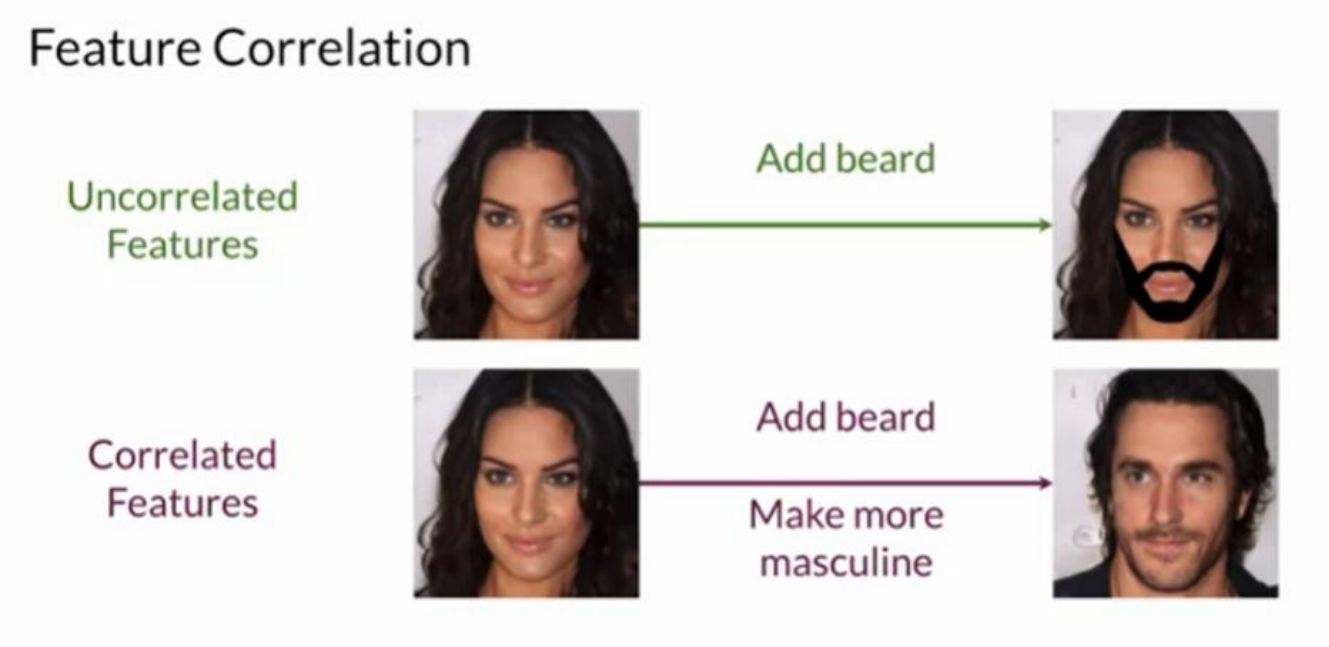
Feature 5

Feature 6

얘네들 각각 i번째가 하나의 특징을 나타낸다고 해도,
Clustering의 축은 수직이다.

이 Latent Vector의 각 좌표의 축이
특정 성질을 의미하는 것이 맞을까??

목표/문제 정의 전에 좀 더 살펴봐야할 것



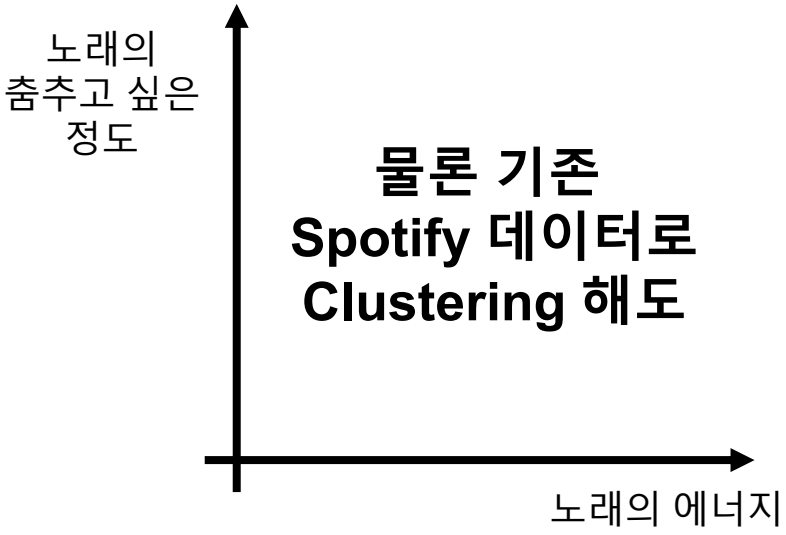
Entaglement 문제!!

여자에게 수염을 붙이도록
feature를 추가하고 싶었는데,

그런 feature 값을 올리자,
아예 남자가 되었더라!!

수염과 남자가 너무 관련 있는 feature였다!

문제 정의



축이 수직처럼 보여도

실제로 축처럼 서로 독립적이지 않다!

Clustering을 안 하더라도

성질들이 서로 분리되어 있지 않다!

Latent Vector를 대상으로 Clustering을 할 것이지만, 그 전에

우리가 한번 Latent Vector의 각 좌표축이 분리된 성질을 갖도록 해보자!

목표 정의

Entaglement를 깨버리는

Disentanglement를 첨가한 Latent Vector를 만들어
더 질이 좋은 Clustering을 해보자.

1. 중간 목표) Clustering을 하기 전에 이미 **성질이 분리된 Latent Vector**는 만들어졌다.
이런 Latent Vector를 이용한 추천시스템 Similarity나 성능이 잘 나오는지 보자.

2. 최종 목표) 이러한 **Latent Vector를 사용한 Clustering을 통해**
기존의 Spotify 성질로 Clustering하는 것보다 성능이 좋은지 관찰하고,
기존의 Spotify 성질로 Clustering하는 것에서 관찰 할 수 없던 것이 있는지 살펴본다.

제안 기법 및 알고리즘

Disentanglement

근데 이 Disentanglement.
GAN(Generative Adversial Network)에서 쓰이는 방식인 것 같은데...

기존의 Disentanglement 방식으로는
우리가 원하는 Clustering에는 안정적이지 않다고 판단!

2.3. Orthogonal Regularization

Many recent studies have been given to incorporate the orthogonality for improving deep network training [5, 19, 31, 36, 43, 44]. Wang *et al.* [43] imposed orthogonal regularization on the weighting parameters with the form $\|\mathbf{W}^T\mathbf{W} - \mathbf{I}\|_2$, where $\mathbf{W}$ is the weight matrix and $\mathbf{I}$ is an identity matrix. Jia *et al.* [19] encouraged the orthogonality by bounding the singular values of the weight matrix in a narrow range around 1. For improving image generation quality, BigGAN [4] introduced a "truncation trick" by removing the diagonal terms from the regularization. Bansal *et al.* [1] introduced another orthogonal regularization by considering both $\|\mathbf{W}^T\mathbf{W} - \mathbf{I}\|_2$ and $\|\mathbf{W}\mathbf{W}^T - \mathbf{I}\|_2$.

다행히 **orthogonality를 이용하여**
Disentanglement적인 해석이 되는 안정적인 방식이 있다!

Yuxiang Wei. (2021). Orthogonal Jacobian Regularization for Unsupervised Disentanglement in Image Generation

Disentanglement

좌표축1	0.3	0.2	0.4
좌표축2	0.1	0.8	0.2

Weight

x

0.3
0.2
0.8

Input

bias는 제외

=

0.45
0.35

Latent Vector

좌표1
좌표2라고
생각할 수 있지 않을까??

1. 이 Latent Vector의 두 요소

2. 사실 Input Vector에
이 두 행 벡터를 통해 만들어진 것이다

3. 그렇다면 Latent Vector의 이 두 요소는
Weight의 두 행 벡터를 성질로 가진다고 볼 수 있지 않을까?

4. 그렇다면 저 **Weight의 행 벡터들을 수직으로 만든다면**, 축이 수직이 되는 것처럼
Latent Vector의 요소들이 서로 독립된 성질을 갖는다고 할 수 있지 않을까?

Disentanglement

어떻게 Weight의 열 벡터를 수직으로 만들까?

W 1행	0.2	0.3
W 2행	0.4	0.5

$\boldsymbol{W}$

x

W 1행	W 2행
0.2	0.4
0.3	0.5

$\boldsymbol{W^T}$

0.13 = 1행 x 1행	0.23 = 1행 x 2행
0.23 = 2행 x 1행	0.41 = 2행 x 2행

$\boldsymbol{WW^T}$

행렬의 각 행끼리 곱해진 방식

다른 행끼리 곱해진건 0에 가깝게 해야 한다!

Disentanglement

$$\left\| \begin{array}{|c|c|} \hline 0.13 = 1열 \times 1열 & 0.23 = 1열 \times 2열 \\ \hline 0.23 = 2열 \times 1열 & 0.41 = 2열 \times 2열 \\ \hline \end{array} - \begin{array}{|c|c|} \hline 1 & 0 \\ \hline 0 & 1 \\ \hline \end{array} \right\|^2 \longrightarrow 0$$

WW^T $\qquad$ I

그렇다면 이 식을 I에 최대한 가깝게 해보면 어떨까?

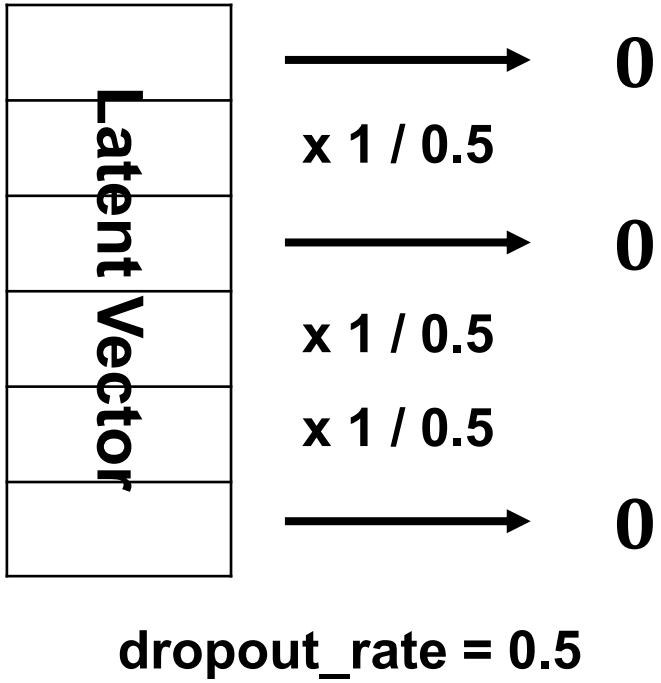
다른 행 벡터끼리의 곱은 0으로 만들고, 자기 자신의 크기는 1로 한다.

```
W_Encoder = model.encoder.weight
H = W_Encoder.size(0)

gram_matrix = torch.matmul(W_Encoder, W_Encoder.T)
I = torch.eye(H)
loss_orthogonality = LA.matrix_norm(gram_matrix - I)**2

loss = loss + lambda_orthogonality * loss_orthogonality

loss.backward()
optimizer.step()

running_loss += loss.item()
```

이 과정을 loss에 일정 부분 넣고,
역방향 계산에 포함되게 하자!!

이를 epoch별로 반복하자!

Denoising AutoEncoder



Latent Vector의 항들을
dropout_rate의 확률로 0으로 만들어버리자!

중요한 항이였다면 어차피 학습하며 복구될거고,

쓸데 없는 noise였다면 복구 안 될 것이다.

Weight의 성질 독립에 기여할 것으로 기대
noise였던 열 벡터 성질은 축소되는 식으로.

```
class Net(nn.Module):
    def __init__(self, D, K, dropout_rate=0.5):
```

```
self.dropout = nn.Dropout(p=dropout_rate)
```

```
def forward(self, x):
```

```
x = self.dropout(x)
```

Activation Function

Input **Hidden** **Output**

ReLU:

Latent Vector에 0이 많도록 설정.
성질 분리에 기여함

DAE가 0으로 만들었던 것처럼,
ReLU도 0으로 만들었는데
중요하면 복원이 되겠지!

Sigmoid:

평점 행렬을 0부터 1 사이로 만들어
Output도 0부터 1 사이의 값이
나오게 한다.

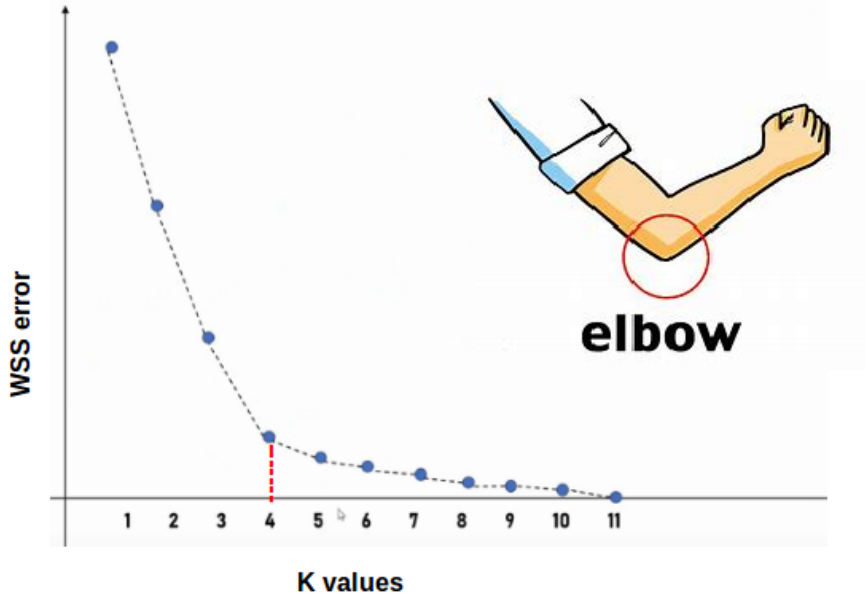
Clustering 기법

K-means랑 **Elbow Method**으로 물론 할 것이지만,

이미 Weight의 축이 각각 수직으로.
서로서로 독립된 성질을 갖는 상황이 됐으니,

각 Embedding Vector에서 제일 큰 요소로 Clustet를 보내면 어떨까 하는 궁금증!

Elbow method




```
dominant_factors = torch.argmax(track_representations, dim=1)

ortho_clusters = defaultdict(list)
for track_id, factor_id in enumerate
    ortho_clusters[factor_id.item()]
ortho_clusters = dict(ortho_clusters
```

```
수직 요소 기반 클러스터링: 총 32 클러스터
  클러스터 1번: 388 트랙
  클러스터 2번: 228 트랙
  클러스터 3번: 842 트랙
  클러스터 4번: 301 트랙
  클러스터 5번: 1098 트랙

가장 원소의 개수가 작은 클러스터는 2번 : 228 tracks
```

생각보다 잘 모여있었다!
한번 평가해보자!

Clustering 기법

1번: Latent Vector를 통한 Elbow Method + K-means Clustering

2번: Latent Vector별 가장 큰 값을 가진 좌표의 Cluster

3번: 2번의 방식을 Elbow Method의 최적의 k로 Cluster 줄이기

독립된 좌표축이라 유사도가 거의 없겠지만,
그나마 유사한 순으로.

데이터 수집 및 평가 지표

데이터 수집 상황

Spotify API: 몇번 이상의 데이터 호출 시, 7시간 ~ 24시간 호출을 막음.
우리가 원하는 데이터 양을 확보하는 데 무리라고 판단.

Zenodo: 대용량의 파일을 확보하는 데 성공했으나,
많은 데이터가 누락되었고, 특히 track_id가 누락되어 상호작용이 어려움

Kaggle: 데이터 양은 좋으나, 누락된 데이터가 많았다.
좋은 데이터가 있었어도 Spotify에게 직접 삭제되었던 경우가 많다.

그래도 Kaggle에서 상호작용되는 데이터를 확보하는 데 성공했고,
데이터를 clean하게 하는 데 성공했다!

100만 이상의 rating과 track 정보 확보

다만 음원의 성질이 존재하지 않아,
비교할 성질 기반 K-means Clustering은 다른 Track Data를 사용하기로 했다.

평가 지표

(중간 목표)추천시스템 성능: 기본적으로 AutoEncoder니까 **MSE(RMSE)** 지표가 중요!

track의 Latent Vector를 구하고 싶어,
Item Based가 되도록 Input을 넣었지만,

Item Based가 이유가 아닐 수 있지만,
실제로 Item Based로 했을 때는 평가 지표가 심각하게 낮게 나와,
User Based로 해야 하는 것이 맞다고 판단했다.

NDCG랑 **Hit Rate**를 구하기 위해서는
Input이 User Based가 되도록 하여 구했다.

유저의 축을 수직으로 만드는거라
알고리즘에는 큰 문제가 없다.

(최종 목표)Clustering 성능: 본래 분산도(inertia)를 기준으로 하려고 했으나,

```
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
```

Clustering에 쓸 수 있는 유용한 라이브러리가 있어 써보았다.

그 외에 우리의 힘으로 구현할 수 있는
변동계수, **지니계수**랑 **아티스트가 얼마나 다양한지**를
추가 평가지표로 썼다.

실험 결과

Loss 확인

```
Epoch 9/50: Train Loss: 0.0039, Validation Loss: 0.0012, Orthogonality Loss: 0.003374
Epoch 10/50: Train Loss: 0.0038, Validation Loss: 0.0012, Orthogonality Loss: 0.003925
Epoch 11/50: Train Loss: 0.0037, Validation Loss: 0.0011, Orthogonality Loss: 0.003399
Epoch 12/50: Train Loss: 0.0036, Validation Loss: 0.0011, Orthogonality Loss: 0.004112
Epoch 13/50: Train Loss: 0.0036, Validation Loss: 0.0010, Orthogonality Loss: 0.012926
Epoch 14/50: Train Loss: 0.0037, Validation Loss: 0.0010, Orthogonality Loss: 0.022441
Epoch 15/50: Train Loss: 0.0037, Validation Loss: 0.0010, Orthogonality Loss: 0.028501
```

수직성을 만드는 $\|WW^T - I\|^2$가
전혀 줄어들지 않는 모습

Train Loss 위주로 가다가,
기울기 변화가 Orthogonality Loss 쪽으로 간 것으로 추정

```
Epoch 27/50: Train Loss: 50.4882, Validation Loss: 0.0010, Orthogonality Loss: 2895.822021
Epoch 28/50: Train Loss: 1.7590, Validation Loss: 0.0009, Orthogonality Loss: 0.007682
Epoch 29/50: Train Loss: 0.0033, Validation Loss: 0.0009, Orthogonality Loss: 0.000016
```

그러나 한번의 Loss 폭발 이후 안정화되는 모습!!

```
Epoch 50/50: Train Loss: 0.0032, Validation Loss: 0.0008, Orthogonality Loss: 0.000000
학습 완료!
```

최종적으로 아주 적은 Loss가 완성되었다!!

직교성 확인

```
순위 | Norm
--------------------
  1 | 1.0000 (상위)
  2 | 1.0000 (상위)
  3 | 1.0000 (상위)
  4 | 1.0000 (상위)
  5 | 1.0000 (상위)
  6 | 1.0000 (상위)
  7 | 1.0000 (상위)
  8 | 1.0000 (상위)
  9 | 1.0000 (상위)
 10 | 1.0000 (상위)
...
 28 | 1.0000 (하위)
 29 | 1.0000 (하위)
 30 | 1.0000 (하위)
 31 | 1.0000 (하위)
 32 | 1.0000 (하위)
```

행 벡터의 Norm이 1에 가깝게 나오고,

```
최종 직교 오차: 0.000000
W x W^T와 I 중 가장 차이 큰 값: 0.000077

W x W^T 6 x 6까지:
[[ 1.      0.      0.      0.      0.     -0.    ]
 [ 0.      1.      0.      0.      0.     -0.    ]
 [ 0.      0.      1.      0.      0.     -0.    ]
 [ 0.      0.      0.      1.      0.     -0.    ]
 [ 0.      0.      0.      0.      1.0001  0.    ]
 [-0.     -0.     -0.     -0.      0.      1.    ]]
```

최종 오차가 아주 작게 나오고,

WW^T 가 거의 I에 가까운 모습이다!

추천도 확인

track_id 18930 의 유사도 기반 추천

	track_id	artist_name	track_name	playlistname	\ similarity
25680	18959	Infected Mushroom	Artillery	Starred	0.915831
445	5125	Pendulum	Granite	Electro	0.907844
312	25216	Daft Punk	Adagio For Tron	Daft Punk	0.897694
507	5129	Pendulum	The Tempest	Electro	0.889936
405	5124	Pendulum	9,000 Miles	Electro	0.882256
748	22455	Pendulum	Crush	Starred	0.874035
817	9470	Pendulum	The Island - Pt. II (Dusk)	Starred	0.871952
138115	10853	Wolfgang Gartner	Illmerica	PARTY!	0.866168
505	5128	Pendulum	The Other Side	Electro	0.859020
339	25217	Daft Punk	End Of Line	Daft Punk	0.858334

최고 유사도가 0.7 근처가 나오는 경우도 있지만, 0.9도 넘게 나오는 모습!

Overfitting이 의심되는 유사도가 나오긴 한다.

플레이리스트 안의 것을 유사하게 보는 경향이 있다.

중간 목표) 추천시스템 성능

```
>>> Model RMSE: 0.0573
Calculating NDCG@10 on 500 users...
Processed 100/500 users...
Processed 200/500 users...
Processed 300/500 users...
Processed 400/500 users...
Processed 500/500 users...
>>> Model NDCG@10: 0.0504
{'hit_rate@10': 0.22492926752593523}
```

User Based로 바꾸어 평가지표를 적용하였으나,

본래 AutoEncoder의 기능이 복원에 치중되어 있어
RMSE는 좋게 나왔지만,

추천이 적중하였는지 체크하는 **hit rate**와 **NDCG**,
그 중에서도 추천 순위도 체크하는 NDCG는
아주 작게 나온 것으로 확인되었다.

당연히 NCF보단 수치가 나쁘게 나올 것이고,

일단 최종 목표인 Clustering을 확인해보기로 했다.

최종 목표) 클러스터링 확인

```
--- 실루엣 계수 (높은게 좋음) ---
Method 1 (K-means Latent):     0.1407
Method 2 (Orthogonal Max):     -0.1641
Method 3 (Orthogonal): -0.1533
승자는 ✓ K-means Latent

--- Davies-Bouldin Index (작을수록 좋음) ---
Method 1 (K-means Latent):     2.0414
Method 2 (Orthogonal Max):     4.8254
Method 3 (Orthogonal): 5.9441
승자는 ✓ K-means Latent

--- Calinski-Harabasz Index (높을수록 좋음) ---
Method 1 (K-means Latent):     2123.6513
Method 2 (Orthogonal Max):     72.5982
Method 3 (Orthogonal): 81.5910
승자는 ✓ K-means Latent
```

심지어 마이너스

```
--- 클러스터 상세 지표 ---

Method 1 (K-means Latent):
  Size range: 248 - 13942
  Mean ± Std: 3477.2 ± 4163.0
  CV: 1.1972
  Gini: 0.5486

Method 2 (Orthogonal Max):
  Size range: 58 - 5120
  Mean ± Std: 869.3 ± 1283.0
  CV: 1.4758
  Gini: 0.6209

Method 3 (Orthogonal):
  Size range: 110 - 17948
  Mean ± Std: 3477.2 ± 5769.6
  CV: 1.6592
  Gini: 0.7069

--- 변동 계수 (작을수록 좋음) ---
승자는 ✓ K-means Latent

--- 지니 계수 (작을수록 좋음) ---
승자는 ✓ K-means Latent

--- 아티스트 다양성 (higher is better) ---
Method 1 (K-means Latent):     0.3438
Method 2 (Orthogonal Max):     0.5307
Method 3 (Orthogonal): 0.4319
승자는 ✓ Orthogonal Max
```

클러스터링 지표 라이브러리를 대부분 써본 결과,

3가지 Clustering 방식 중 Latent Vector K-means 방식이 매우 우위에 있었으며,

아티스트 다양성에 대해서만 가장 큰 값 좌표 Cluster 방식이 우위에 있었음을 확인했다.

최종 목표) 클러스터링 확인

```
--- 실루엣 계수 (높은게 좋음) ---
Method 1 (K-means Latent):      0.1407

--- Davies-Bouldin Index (작을수록 좋음)
Method 1 (K-means Latent):      2.0414

--- Calinski-Harabasz Index (높을수록 좋음) --
Method 1 (K-means Latent):      2123.6513

--- 클러스터 상세 지표 ---

Method 1 (K-means Latent):
  Size range: 248 - 13942
  Mean ± Std: 3477.2 ± 4163.0
  CV: 1.1972
  Gini: 0.5486

--- 아티스트 다양성 (higher is better)
Method 1 (K-means Latent):      0.3438
```

```
최적의 k는  5
✓ 완료! 총 5 클러스터
   분산도: 11135.37

--- 실루엣 계수 ---
다른 데이터셋 K-Means:      0.2728

--- Davies-Bouldin Index ---
다른 데이터셋 K-Means:      1.1983

--- Calinski-Harabasz Index ---
다른 데이터셋 K-Means:      37819.7473

--- 클러스터 상세 지표 ---
  다른 데이터셋 K-Means:
  Size range: 6539 - 27149
  Mean ± Std: 17902.6 ± 7544.1
  CV: 0.4214
  Gini: 0.2304

--- 아티스트 다양성 ---
다른 데이터셋 K-Means:      0.3857

✓ 평가 지표 완료
```

기존 Dataset에는 음원의 특징이 담겨있지 않아, **음원의 특징**이 있는 **다른 Dataset**으로 **K-means Clustering**을 진행해보았다.

아쉽게도 모든 면에서 음원의 특징을 이용한 Clustering이 우수하다는 결론이 나왔지만,

음원의 특징이 주어지지 않은 채로 Clustering에 성공했다는 데에 의의가 있다.

한계점 및 개선점

Input 행렬의 Sparse 문제

```
MemoryError: Unable to allocate 198. GiB for an array with shape (1670049, 15910) and data type float64
```

필터링을 하나도 안 했을 때 생기는 에러. 필터링을 약하게 해도 생기곤 한다.

사실 이 모델과 Clustering을 만드는 과정에서 **Dataset을 많이 줄였다.**

음원 Dataset에 행렬을 넣는다는 것 자체부터 엄청나게 sparse하기 때문!

SVD로 하려 했던 것과 같은 문제!
대규모 클러스터링은 힘들 수 있다!

제대로 모델을 사용하려면 sparse 하지 않은 행렬을 사용하거나,
아예 CPU 환경에서 학습하게 하는 방법이 있을 듯하다.

Epoch와 폭파 문제

```
Epoch 224/300: Train Loss: 0.1164, Validation Loss: 0.0691, Ortho Loss: 0.123693
Epoch 225/300: Train Loss: 0.1168, Validation Loss: 0.0690, Ortho Loss: 0.114608
Epoch 226/300: Train Loss: 59.3635, Validation Loss: 0.0690, Ortho Loss: 40.097134
Epoch 227/300: Train Loss: 0.1576, Validation Loss: 0.0687, Ortho Loss: 0.036589
Epoch 228/300: Train Loss: 0.1141, Validation Loss: 0.0686, Ortho Loss: 0.000266
Epoch 229/300: Train Loss: 0.1142, Validation Loss: 0.0685, Ortho Loss: 0.000243
Epoch 230/300: Train Loss: 0.1142, Validation Loss: 0.0684, Ortho Loss: 0.000163
Epoch 231/300: Train Loss: 0.1137, Validation Loss: 0.0684, Ortho Loss: 0.000151
Epoch 232/300: Train Loss: 0.1133, Validation Loss: 0.0683, Ortho Loss: 0.000133
```

Epoch를 많이 넣으면, 언젠가 또 Loss가 올라 한번 더 폭파한다!

좋게 보면, Loss가 올라도 다시 자정 작용이 생기는 것이고,
나쁘게 보면, 모델이 불안정하다는걸 시사한다.

안정적인 값을 얻으려면 Epoch 값을 많이 늘리고,
Loss가 폭발 수치가 되면 남은 Epoch를 5로 하는 방법이 있겠다.

Activation Function과 Layer 개선점

현재 Latent Vector가 사실 0부터 1 사이의 값은 아니다!

물론 Clustering을 할 때 그런 값이 필수 조건이 아닌데,
이왕이면 ReLU랑 Sigmoid를 거친 Latent Vector를 가지고 싶다!

Input	Hidden1	Hidden2	Output
	ReLU	Sigmoid	Sigmoid

그렇다면 Multi-Layer는 어떨까?

Activation Function이 아니더라도 Latent Vector의 정확도는 상승할 것이다.
그러나 weight의 역할을 하는 것이 여러 개인 상황!

그렇다면 Joint SVD를 통해 weight를 하나로 해석하면 어떨까?

Multi-Layer를 하면서 모델과 Latent Vector의 정확도를 챙기고,
하나의 순수한 weight 행 벡터 set을 만들어서 basis로 쓰는 방식을 생각해볼 수 있다!

각 팀원 별 역할

Tsoy Alina Vladimirovna: 데이터 수집 및 전처리

공정한: 모델 구현 및 팀원 코드 통합

정우찬: 클러스터링 구현 및 모델 구현 보조

Zaripov Damir: 평가지표 구현

여담

나중에 안 사실로는,

우리가 한 Clustering 방식은
일종의 **Deep Clustering**이라고 하는 것 같으며,

Disentanglement적인 방식은
정확히는 **직교 정규화**라고 하는 것 같다.

내용이 궁금해졌다면 찾아보자!!

들어주셔서 감사합니다!